



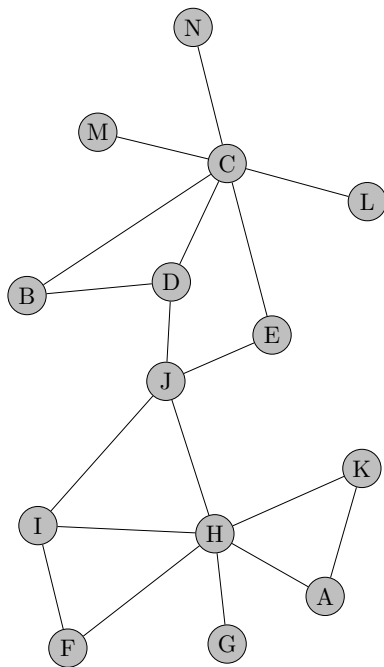
Name: _____ Anzahl Zusatzblätter: _____ Punkte: _____

Alle schriftlichen Unterlagen sind erlaubt, elektronische Hilfsmittel sind verboten! Als Zusatzblätter sind ausschließlich A4-Blätter mit ordentlichen Kanten zulässig, da nur diese vom Mehrblatteinzug des Scanners korrekt verarbeitet werden. Die Verwendung von Bleistiften ist erlaubt, rote Farbe ist jedoch zu vermeiden.

Aufgabe 1 (10 Punkte) Breiten- und Tiefensuche

Führen Sie die Algorithmen Breitensuche (BFS) und Tiefensuche (DFS) auf dem Graphen G_1 aus, wobei jeweils mit Knoten D gestartet wird. Geben Sie in der Tabelle rechts die Entdeckungs- und Fertigstellungszeiten ($\tau_d(v)$, $\tau_f(v)$ für $v \in V$) an.

Graph G_1 :



Schritt	BFS		DFS	
	entdeckt	abgeschlossen	entdeckt	abgeschlossen
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
22				
23				
24				
25				
26				
27				
28				

Aufgabe 2 (8 Punkte) Klammerprüfung mit Stack

Kurzreferenz – Die Methode `Deque<Character>` (aus `java.util`) kann als Stack verwendet werden:

<code>stack.push(x)</code>	Legt Element x oben auf den Stack.
<code>stack.pop()</code>	Entfernt das oberste Element und liefert es zurück.
<code>stack.peek()</code>	Liefert das oberste Element, ohne es zu entfernen.
<code>stack.isEmpty()</code>	Liefert true , wenn der Stack leer ist.

Gegeben ist folgender Java-Code. Die Methode `checkBrackets` soll prüfen, ob alle Klammern in einem String korrekt geschachtelt sind.

Regeln:

- Erlaubte Klammerarten: (), [], { }
- Alle anderen Zeichen (Buchstaben, Zahlen, Operatoren...) werden ignoriert
- Jede öffnende Klammer muss von der **passenden** schließenden Klammer geschlossen werden
- Die Schachtelung muss stimmen: ([]) ist **falsch**, {[()]} ist **richtig**

```
1  import java.util.ArrayDeque;
2  import java.util.Deque;
3
4  public class BracketChecker {
5
6      /**
7       * Prüft, ob alle Klammern im String korrekt
8       * geschachtelt sind.
9       * Erlaubt: ( ), [ ], { }
10      * Alle anderen Zeichen werden ignoriert.
11      *
12      * @return true wenn alle Klammern korrekt geschachtelt
13      *         false sonst
14      */
15     public static boolean checkBrackets(String input) {
16         Deque<Character> stack = new ArrayDeque<>();
17
18         for (char c : input.toCharArray()) {
19
20             // TODO 1: Öffnende Klammern auf den Stack legen
21             // ... Ihre Lösung ...
22
23
24
25             // TODO 2: Bei schliessender Klammer prüfen:
26             // - Ist der Stack nicht leer?
27             // - Passt das oberste Element zur
28             //   schliessenden Klammer?
29             // Falls nicht: return false
30             // ... Ihre Lösung ...
31
32         }
33
34         // TODO 3: Am Ende prüfen, ob alle Klammern
35         // geschlossen wurden
36         // ... Ihre Lösung ...
37
38         return false; // Platzhalter
39     }
40
41     public static void main(String[] args) {
42         System.out.println(
43             checkBrackets("{[( )]}")); // (a)
44         System.out.println(
45             checkBrackets("[ ( ]")); // (b)
46         System.out.println(
47             checkBrackets("(( ( ))")); // (c)
48         System.out.println(
49             checkBrackets("(a + [b * {c}]]")); // (d)
50         System.out.println(
51             checkBrackets("")); // (e)
```

```
52         System.out.println(  
53             checkBrackets(")test(")); // (f)  
54     }  
55 }
```

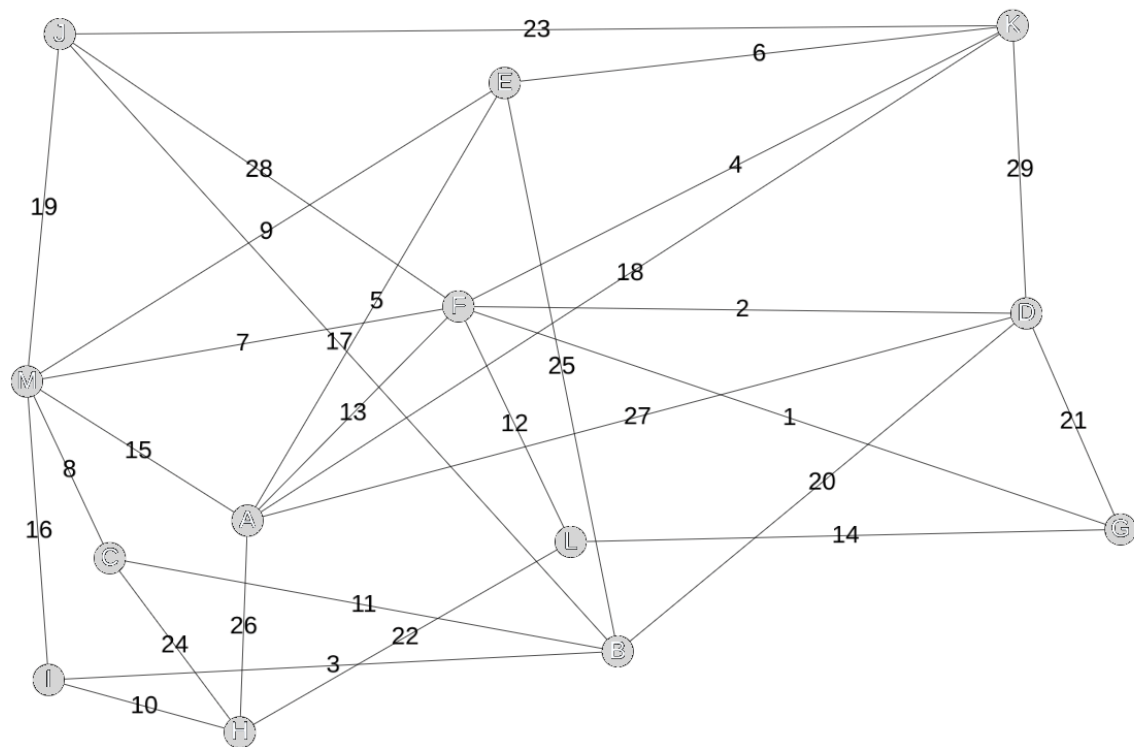
Aufgaben:

- Vervollständigen Sie die drei mit **TODO** markierten Stellen in der Methode **checkBrackets**.
- Die **main**-Methode enthält **einen Syntaxfehler**. Finden und korrigieren Sie ihn.
- Geben Sie für alle sechs Aufrufe **(a) – (f)** in der korrigierten **main**-Methode an, welches Ergebnis jeweils ausgegeben wird:

	true	false
checkBrackets("{ [()] }")	☐	☐
checkBrackets("([])")	☐	☐
checkBrackets("((())")	☐	☐
checkBrackets("(a + [b * {c}]")	☐	☐
checkBrackets()	☐	☐
checkBrackets(")test(")	☐	☐

Berechnen Sie mit dem Algorithmus von Dijkstra für G_1 den kürzesten Weg vom Knoten J zum Knoten H . Der Algorithmus kann beendet werden, sobald der Zielknoten fertiggestellt wird.

Graph G_1 :

[illegible]

Bitte wenden!

Aufgabe 4 (8 Punkte) Algorithmen-Design & Effizienz

Szenario: Ihr seid als Berater-Team für ein Start-up engagiert worden, das eine Foto-App entwickelt. Die Nutzerbasis wächst schnell, und die aktuelle Software wird spürbar langsamer.

Teil 1: Der Komplexitäts-Detektiv (Fehlersuche & versteckte Kosten)

Ein Entwickler hat zwei Funktionen geschrieben, um Duplikate in einer Liste von Bild-IDs zu finden. Er behauptet, beide seien „effizient genug“.

Snippet A:

```
1 public static List<Integer> findeDuplikateA(  
2     List<Integer> bildIds) {  
3     List<Integer> duplikate = new ArrayList<>();  
4     // bildIds hat die Laenge n  
5     for (int i = 0; i < bildIds.size(); i++) {  
6         for (int j = i + 1; j < bildIds.size(); j++) {  
7             if (bildIds.get(i).equals(bildIds.get(j))) {  
8                 // Achtung: Die folgende Zeile durchsucht  
9                 // die Liste 'duplikate' komplett!  
10                if (!duplikate.contains(bildIds.get(i))) {  
11                    duplikate.add(bildIds.get(i));  
12                }  
13            }  
14        }  
15    }  
16    return duplikate;  
17 }
```

1. **Die Analyse:** Der Entwickler sagt, Snippet A habe eine Laufzeit von $O(n^2)$.
 - Untersucht die Zeile `if (!duplikate.contains(...))`.
 - Wie wirkt sich diese Zeile auf die tatsächliche Komplexität aus, wenn sehr viele Duplikate gefunden werden? Begründet eure Entscheidung.
2. **Die Alternative:** Hier ist Snippet B. Bestimmt die Komplexität dieser Funktion und erklärt kurz, warum dieses Snippet bei 1.000.000 Bildern deutlich performanter ist als Snippet A.

Snippet B:

```
1 public static List<Integer> findeDuplikateB(  
2     List<Integer> bildIds) {  
3     Collections.sort(bildIds); // wie Mergesort:  $O(n \log n)$   
4     List<Integer> duplikate = new ArrayList<>();  
5     for (int i = 0; i < bildIds.size() - 1; i++) {  
6         if (bildIds.get(i).equals(bildIds.get(i + 1))) {  
7             if (duplikate.isEmpty()  
8                 || !duplikate.get(duplikate.size() - 1)  
9                 .equals(bildIds.get(i))) {  
10                duplikate.add(bildIds.get(i));  
11            }  
12        }  
13    }  
14    return duplikate;  
15 }
```

Teil 2: Real-World Szenario (Skalierung & Entscheidung)

Eure App bekommt eine neue Funktion: „**Personenerkennung**“. Ihr müsst entscheiden, wie die Datenbank der Gesichter durchsucht wird.

- **Algorithmus „Flash“** $O(n)$: Sucht simpel (sequenziell), braucht aber kaum zusätzlichen Speicher.
- **Algorithmus „SmartIndex“** $O(\log n)$: Nutzt einen komplexen Index (z. B. eine Baum-Struktur). Das Erstellen des Index dauert einmalig lange.

3. **Die Skalierungs-Tabelle:** Füllt die Tabelle mit der (theoretischen) Anzahl an Operationen aus:

Nutzer (n)	Flash $O(n)$	SmartIndex $O(\log n)$	Faktor (wie viel schneller?)
1.000	1.000	~ 10	100-mal schneller
1.000.000			
1.000.000.000			

4. Die „**BusinessEntscheidung**“: Das Start-up hat aktuell nur 500 Nutzer pro Gerät. Ein Investor sagt: „Nehmt den einfachen Algorithmus *Flash*, der reicht völlig!“

- Wann würdet ihr ihm widersprechen?
- Gibt es einen Punkt, an dem der „schnellere“ Algorithmus $O(\log n)$ in der Praxis schlechter sein könnte?
(Tipp: Denkt an den Aufwand für die Pflege des *Index*.)

Teil 3: Transfer (Kreatives Design)

Stellt euch vor, ihr habt eine **sortierte** Liste von 1.000.000 Telefonnummern. Ihr sucht eine bestimmte Nummer.

5. Beschreibt (in Worten oder Pseudocode) ein Verfahren, das **nicht** jede Nummer einzeln prüft, sondern die Sortierung ausnutzt.
6. Welche Komplexitätsklasse (Big O) hat dieses Verfahren?
7. Was passiert mit der Effizienz eures Verfahrens, wenn die Liste plötzlich **unsortiert** ist? Was müsstet ihr tun, um euer Verfahren wieder anwenden zu können?

Beurteilungsschlüssel für p Punkte: $p \leq 20$: -, $20 < p \leq 25$: -/~, $25 < p \leq 30$: ~, $30 < p \leq 35$: ~/+, $p > 35$: +;

Viel Erfolg!